
Lab 2: Sensor/Actuators Bring-up

Diagrams & Integration Writeup

18-649 — Distributed Embedded Systems

Weerachai Sang Andrew Noble John Buttlers

RI

RI

ECE

asangium

anoble

jbuttlers

1 System Block Diagram

The wheel streams to the laptop proxy, the proxy sends UDP to the Pi, the Pi forwards one 13-byte UART command per UDP update, and a single NUCLEO-F401RE running Zephyr drives every actuator and returns a 20 ms status frame. Figure 1 shows the hardware and, inside the STM32, the interrupt and thread structure that §4 schedules.

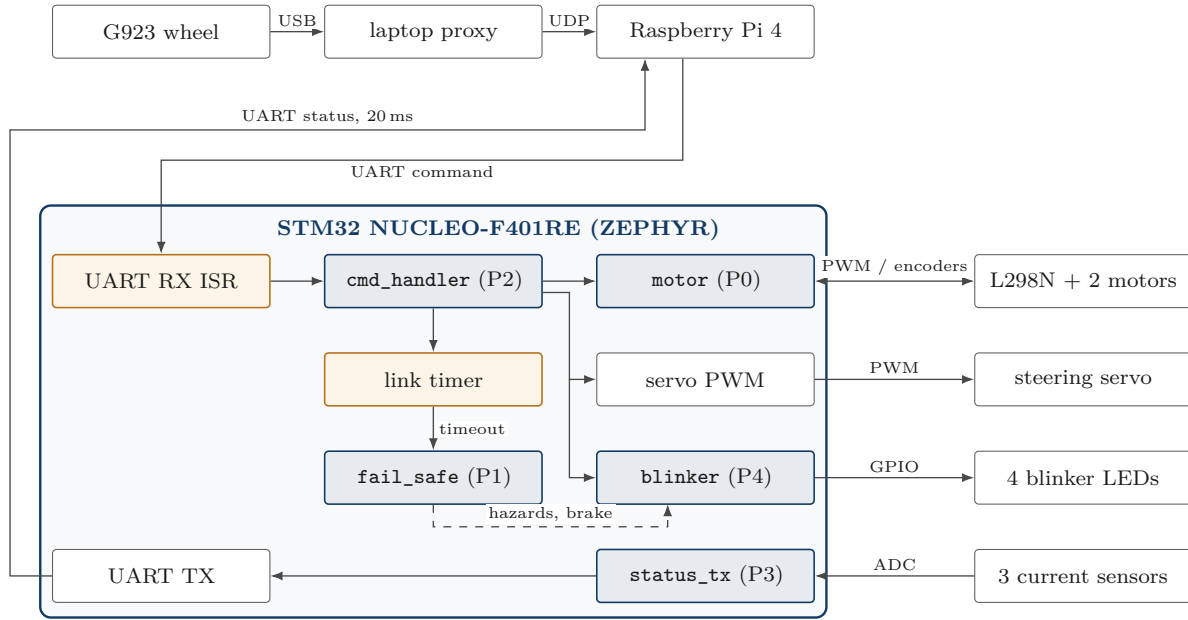


Figure 1. System block diagram. Blue boxes are threads (priority in parentheses), yellow boxes are interrupt and timer-callback contexts. Solid arrows are data or wake-ups. The dashed arrow is the fail-safe override.

2 Circuit Schematic

The bench has four supplies and one common ground: the 12 V wall adapter (motors), a 5 V linear regulator fed from 12 V (servo and current sensors), and the laptop USB through the Nucleo's ST-LINK (Nucleo and its 3.3 V rail, which powers the encoders). Every ground is tied to one breadboard ground rail. Figure 2 draws the rails and Figure 3 draws every signal connection.

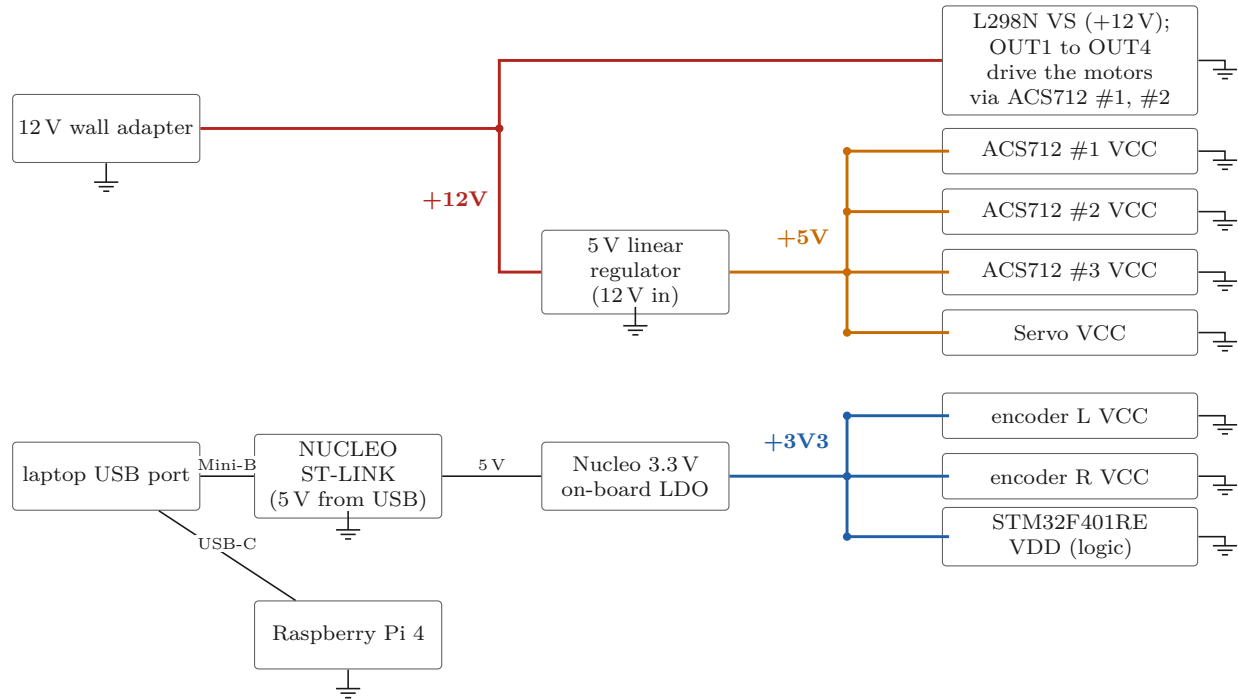


Figure 2. Power rails and ground. All ground symbols are one node. The motors are not on a rail: the L298N outputs switch 12 V to them through ACS712 #1 and #2. The Nucleo's own 5 V is not used by any peripheral.

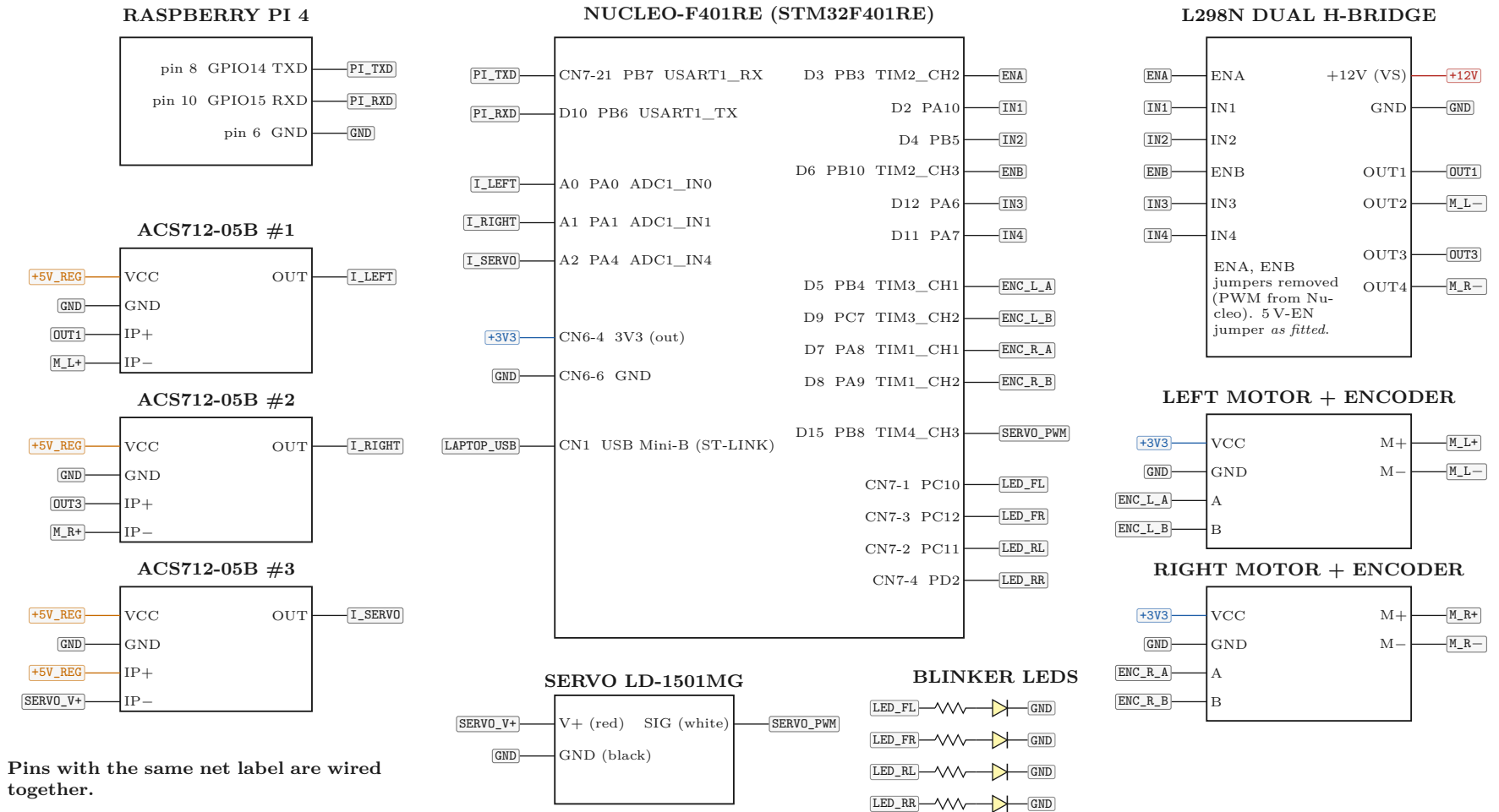


Figure 3. Signal schematic of the dev kit. The ACS712 current inputs are in series with the left motor, right motor and servo supply leads.

3 Updated Team-Defined Values

Shaded rows changed since the Lab 1 table. Each value is the one in the code, with its symbol in `src/config.h`.

ID	Parameter	Value used in Lab 2	Where set
TD-01	Steering axis and raw range	1X: -32767 full left, 0 centre, $+32766$ full right (macOS proxy). Dead zone ± 500 holds centre.	STEER_RAW_MIN/MAX, STEER_DEAD_ZONE
TD-02	Throttle axis and raw range	1Y: 0 released, 32767 fully pressed; first 1 % of travel ignored	THROTTLE_RAW_*, THROTTLE_ZERO_PERCENT
TD-03	Brake axis and raw range	1Rz: 30454 released, 32767 fully pressed (measured); valid range 0 to 32767	BRAKE_RAW_*
TD-04 / 05	Left / right turn button	rgbButtons[5] left, rgbButtons[4] right (SDL numbering); bits 1, 2 of frame byte 10	bridge.c, BUTTON_LEFT/RIGHT
TD-06	Self-test button	rgbButtons[10]; bit 0 of frame byte 10. One press enters; two presses within 500 ms exit. Debounce: a release must be seen for 30 ms	BUTTON_SELF_TEST, SELF_TEST_EXIT_*
TD-07	UDP state-update interval	50 ms (<code>-interval-ms 50</code>); one UART command per update	proxy command line
TD-08	Turn-signal arm threshold	$ \text{steer} \geq 1000$ raw (3 % of full lock) in the selected direction	BLINKER_TURN_THRESHOLD
TD-09	Turn-signal cancel threshold	return inside the ± 500 dead zone (or cross it between frames)	STEER_DEAD_ZONE
TD-10	Throttle to velocity, V_{max}	linear, monotonic: up to 1 % coasts, then 2 to 100 % $\rightarrow$ 4 to 200 RPM wheel speed; $V_{max} = 200$ RPM (unloaded hardware maximum about 317 RPM). Each wheel has its own PID on a shared target; reported velocity is the mean of the two encoders	MOTOR_MAX_RPM
TD-11	Steering to servo mapping	linear outside the dead zone, monotonic: $-32767 \rightarrow 1000 \mu\text{s}$, $0 \rightarrow 1500 \mu\text{s}$, $+32766 \rightarrow 2000 \mu\text{s}$. Linkage reaches its stops inside this range; angle <i>to measure</i>	SERVO_*_US
TD-12	Brake activation threshold	raw ≥ 31610 , the midpoint of the measured pedal travel (50 %)	BRAKE_PRESSED_AT
TD-13	Load-to-force mapping	not used: no force feedback this lab	none
TD-14	Max phase correction per BLINK_SYNC	not used: one MCU, one timer drives all four LEDs	none
TD-15	Motor PWM frequency	1 kHz (Lab 1 proposed ≥ 20 kHz)	overlay, PWM_MSEC(1)
TD-16	Servo frame rate	50 Hz, 20 ms period; pulse limits 500 to 2500 μs	overlay, SERVO_PULSE_MIN/MAX_US
TD-17	Duty limits	0 coasts; any drive maps to 20 to 100 % actual PWM (wheel-up sweep: wheels did not start at 20 %, did at 30 %)	MOTOR_DUTY_OFFSET_*, MOTOR_MAX_DUTY_*
TD-18	Current readings (future fault thresholds)	At rest / free-spinning / brief stall, per sensor: L 400 / 1100 / 2600 mA; R 400 / 1000 / 2500 mA; servo 800 / 1000 / 2000 mA	bench, STATUS log
TD-19	Velocity PID	$K_p = 0.45 \text{ \%}/\text{RPM}$, $K_i = 0.60 \text{ \%}/(\text{RPM s})$, $K_d = 0.020 \text{ \% s}/\text{RPM}$ on measurement, 50 ms derivative filter, 10 ms period, anti-windup by conditional integration. Settling <i>to measure</i>	MOTOR_K*, ENC_PERIOD_MS
TD-20	Measured task WCETs	<i>to measure</i> , GPIO toggle per task (§4)	none
TD-21	Encoder counts per wheel rev	1316 (x4 quadrature; measured 1318.4 L, 1314.0 R over 5 turns); circumference <i>to measure</i>	overlay, counts per revolution

4 Task Table

Zephyr's preemptive scheduler runs the highest-priority ready thread. Priorities are set in `src/config.h` and `prj.conf` for main and logging.

Priority	Task	Period T	Deadline D	Talks to
ISR	USART1 RX ISR	per byte, $86.8\ \mu\text{s}$ at 115200	before the next byte	assembles 13-byte frames and inserts into <code>frame_queue</code>
ISR	<code>control_timer</code>	10 ms	a few μs	gives <code>motor_wake</code>
ISR	<code>link_timer</code>	150 ms after last valid frame	a few μs	gives <code>failsafe_sem</code>
ISR	<code>blinker_timer</code>	500 ms (250 ms hazard)	a few μs	gives <code>blinker_wake</code>
0	<code>motor</code>	10 ms, plus one step per command	2 ms from <code>CMD_RX</code> to <code>PWM_SET</code> / <code>DIR_A</code> (R2.1, R2.2)	encoders (TIM3, TIM1), L298N PWM and IN pins; <code>latest_command</code> and telemetry under a spinlock
1	<code>fail_safe</code>	sporadic, ≥ 150 ms	100 ms to fail-safe outputs (R3, checkoff 7)	<code>failsafe_sem</code> , <code>command_lock</code> mutex, motor (brake), servo (off), blinker (hazard), zone state
1	<code>main</code>	once at boot	none	starts USART1 interrupt and servo; returns
2	<code>cmd_handler</code>	sporadic, 50 ms nominal, 1.13 ms min	2 ms brake / throttle hand-off; 10 ms self-test; 50 ms steering; 100 ms blinker	<code>frame_queue</code> , <code>command_lock</code> , <code>motor_control_submit</code> , servo PWM, <code>blinker_update_turn</code> , <code>link_timer</code> , zone state
3	<code>status_tx</code>	20 ms	2 ms to first TX byte (keeps 20 ms $\pm 10\%$)	ADC1 $\times 3$, zone state (atomic), USART1 TX to Pi, console <code>STATUS</code> line every 100 ms
4	<code>blinker</code>	500 ms (250 ms hazard)	toggle within 5 ms of tick (keeps 50 % duty)	4 LED GPIOs, mode under a spinlock
5	log thread	background	none	deferred <code>printk</code> to USART2 console

5 Integration Writeup: Three Zones on Dual CAN

Functions by zone. The Pi keeps UDP reception and takes over what needs the whole wheel state: scaling raw counts to engineering units (`STEER_RAW_*`, `THROTTLE_RAW_*`, `BRAKE_PRESSED_AT` move from `config.h` to the Pi), button edge detection and debounce, and the blinker state machine, which needs both buttons and the steering angle. The steering zone keeps `servo_driver`, the servo current sensor and the front blinkers. The drivetrain zone keeps `motor`, `motor_driver`, both encoders, both motor current sensors and the rear blinkers. Each STM32 keeps its own `fail_safe` thread and its own hazard outputs.

Lab 2 variable / frame	CAN ID	Message, TX → RX	Change
<code>pi_cmd.brake</code> , brake decision	0x060	DRIVE_CMD, 1 → 3	0 to 100 % and BRAKE_ACTIVE bit
<code>pi_cmd.throttle</code>	0x060	DRIVE_CMD, 1 → 3	0 to 100 %, same frame as brake
<code>pi_cmd.steer</code>	0x080	STEER_CMD, 1 → 2	±1000 instead of raw int16
<code>blinker requested_mode</code> , <code>phase_on</code>	0x0A0	BLINK_SYNC, 1 → 2, 3	pattern + phase + ms into half-cycle
<code>zone_state</code> , <code>link_ok</code>	0x100 to 0x102	HB_*, each → all	20 ms heartbeat replaces the status frame
error entry (self-test, range, link)	0x040	SYS_ERROR, any → all	cause code, so every zone goes to hazards
current mV, RPM, bad-frame count	0x281, 0x282	ZONE_STATUS_n, → 1	100 ms diagnostics
<code>seq</code> , CRC-8	all	every frame	4-bit SEQ; CAN CRC-15 replaces CRC-8

IDs and priority. Lower IDs win arbitration, so the order is safety first: `SYS_ERROR` 0x040, then brake and throttle 0x060 (2 ms budget), steering 0x080 (50 ms), blink sync 0x0A0 (1 ms skew), heartbeats 0x100, diagnostics 0x280. The IDs in this lab (0x100 command, 0x200 status) were placeholders and are renumbered to this catalog. Worst-case CAN transport for `DRIVE_CMD` is 550 μ s, which leaves about 1.4 ms of the 2 ms budget for the Pi and the drivetrain handler; the 3-missed-update link watchdog becomes a 60 ms heartbeat timeout per peer and per bus.

What breaks when the blinker halves are on different MCUs. Today one `blinker_timer` drives all four LEDs, so front and rear cannot drift. Each Nucleo's clock comes from its own ST-LINK 8 MHz source through the PLL, so two boards differ by tens to hundreds of ppm. At 100 ppm the halves drift 0.1 ms per second and break the 1 ms skew limit (R2.4) within about 10 s. Start is also skewed, because each zone receives the mode change at a different time. To mitigate this, each zone runs a local timer, the blink master sends `BLINK_SYNC` every so often with its phase, and each zone corrects by at most 1 ms per sync. Fail-safe hazards need the same care. A zone that loses both buses starts hazards on its own clock, so its phase is only resynchronised when the bus returns.